

Note: This exercise is to be done in **groups**. Submission is a .zip file.

For this exercise, using an Ubuntu Linux-based environment on a real x86_64 computer is necessary. (WSL Ubuntu will work, but Mac systems will not work as expected.)

Many scientific and multimedia applications have a need to apply the same mathematical operations on a lot of data at once (such as calculating dot/cross products and matrix operations). To facilitate this, modern CPUs implement SIMD (Single Instruction, Multiple Data) instructions that allow parallel processing of these operations. However, older CPUs do not necessarily have these instructions, so compilers typically have to generate equivalent assembly language routines to emulate these instructions when running on unsupported CPUs.

You will simulate a simple example of this: perform a multiplication operation on an array of size n with some constant x , using a single function call. (You may recognize this as a “scaling” operation on n -dimensional vectors.)

A. multiplyByX

1. (1 pt) Create a header file called `IntArray.h`, which should contain the following struct declaration. (You may add member functions, but do NOT add more member variables!)

```
struct IntArray {
    int size;
    int *elements;
};
```

2. (2 pts) Create a cpp file called `multiplyByX.cpp`, which contains the following code. Replace the comment in the body with your own implementation. (Do *not* allocate or deallocate any memory in this function!)

```
#include "IntArray.h"

void multiplyByX(IntArray *p, int x)
{
    // multiply x with all elements of IntArray
    // (overwrite the previous elements)
}
```

3. (1 pt) Generate the .s file of `multiplyByX.cpp`, and name it `multiplyByX.s`. Examine the generated assembly code. You could tweak your C++ function so that the generated assembly would be more compact. You may also use the `-O` compiler flag!

4. (2 pts) Create a cpp file called multiplyByXTester.cpp, which contains:

```
#include <iostream>
#include "IntArray.h"

extern void multiplyByX(IntArray *p, int x);

int main()
{
    // - create a new IntArray with your own elements
    //   (don't forget to properly allocate/deallocate the memory!)
    // - print it out
    // - multiply it by a number of your choosing via multiplyByX()
    // - print the result
    return 0;
}
```

5. Test your multiplyByX.s by compiling it with your multiplyByXTester.cpp. (See the Lab 5 slides if you forgot how to do it.) Make sure your code is running correctly before proceeding with the next part of this exercise.

B. multiplyByXGenerator

1. (4 pts) Now it is time for you to write a program that generates your own assembly code! (Think of it as a baby step towards writing your own compiler!)

Create a new cpp file called multiplyByXGenerator.cpp. This should be a standalone program with its own main function (but includes IntArray.h). This program should generate custom assembly code to multiply an IntArray by any positive constant specified by the user (instead of the variable x).

The program should be invoked from the command line with a single parameter specifying the value of x that will be multiplied with the array.

The output of the program is the corresponding assembly code, with the function name multiplyBy<constant>. Pay attention to how the name of the function is written in assembly – there's actually a strict format! (Can you figure it out?)

Example run:

```
csci51@VAMR0:~$ ./multiplyByXGenerator.out 61
.file    "multiplyBy61.cpp"
.text
.globl   _Z12multiplyBy61P8IntArray
.type    _Z12multiplyBy61P8IntArray, @function
_Z12multiplyBy61P8IntArray:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .
        .
        (generate this part of the code as you see fit)
        .
        .
csci51@VAMR0:~$
```

HINT: Modify your multiplyByX.cpp to multiply with a hardcoded constant, say, 61, then generate its assembly. You can use this .s file as your template. You can use the cout function to output the assembly code one line at a time, replacing the relevant parts with your own variables (such as the name of your function, the multiplier constant, etc.)

2. (1 pt) Create a tester program to test your generator, multiplying by 61 as a test case. Name this as multiplyBy61Tester.cpp.

```
#include <iostream>
#include "IntArray.h"

extern void multiplyBy61(IntArray *p);

int main()
{
    // you know what to do...
}
```

3. (1 pt) Generate multiplyBy61.s using your generator program and the standard output redirection operator, >, like this:

```
csci51@VAMR0:~$ ./multiplyByXGenerator.out 61 > multiplyBy61.s
csci51@VAMR0:~$ █
```

4. Ensure that everything works by compiling multiplyBy61Tester.cpp together with multiplyBy61.s and testing the resulting program.

C. No multiply instruction

(4 pts) You will notice that our example uses the constant 61, and if you use g++ to generate the assembly, g++ would use the `imull` instruction. What if you are not allowed to generate the `imull` instruction (or its variants such as `imulq` or `fmul`) in your `multiplyByXGenerator.cpp`?

You may recall in Lab 4 that compilers could optimize your code to avoid expensive operations such as multiplication if faster alternatives exist. Is there a general algorithm by which this conversion can be done? Modify your `multiplyByXGenerator.cpp` to implement this algorithm!

(For this part, if you have to do research, you are required to cite your sources as comments in your source code and in your COA.)

HINT: Your grade will be based on both correctness and optimality! You will get more points if you generate well-optimized code, but you will get big deductions if the code doesn't actually work. Prioritize your effort on perfecting your general conversion method first, before you do further optimizations!

Fun fact: Really old processors (such as the 6502 used in the Apple II and the Nintendo Family Computer, as well as the original ARM1 processor) don't have multiply instructions, so the use case for this isn't very far-fetched!

Submit a zip file containing the following:

- `IntArray.h`
- `multiplyByX.cpp`
- `multiplyByX.s`
- `multiplyByXTester.cpp`
- `multiplyByXGenerator.cpp` (you only need to submit your latest/best version)
- `multiplyBy61Tester.cpp`
- `multiplyBy61.s` (you only need to submit your latest/best version)
- accomplished COA: `Lab5_(surnames in alphabetical order)_COA.pdf`
- name the zip file: `Lab5_(surnames in alphabetical order).zip`

Example submission (the other files are contained within the zip file):

- Lab5_EspinozaManaogSanchez.zip
 - IntArray.h
 - multiplyByX.cpp
 - multiplyByX.s
 - multiplyByXTester.cpp
 - multiplyByXGenerator.cpp
 - multiplyBy61Tester.cpp
 - multiplyBy61.s
 - Lab5_EspinozaManaogSanchez_COA.pdf

This lab will have a **defense component**. (4 pts) You will need to sign up for your defense slot, as discussed by your instructor.